Code and Software Submission Checklist

WrapEvoFS enables auditable feature compression with regret-constrained representative locking

Corresponding author: Huan Zhong
Software release: v0.2.0 reference; v0.3.0 calibration API

Item	Availability and documentation
Software access and execution	
Source code	Installable package and tagged release: `https://github.com/junseoparkX/wrapevofs-package`. Manuscript analysis code and aggregate source data: `https://github.com/junseoparkX/wrapevofs-manuscript-reproducibility`.
Version and license	Reference v0.2.0 and the opt-in calibration extension v0.3.0 are distributed under the OSI-approved BSD 3-Clause License and is available from GitHub and PyPI.
Installation	Install from PyPI with `pip install wrapevofs==0.3.0`. Use v0.2.0 for its frozen reference environment. Source and development installation are documented in the README.
System requirements	Python 3.10–3.12, dependencies, optional selector extras, and CPU execution are documented in the README and package metadata.
Tested versions	Continuous integration tests Python 3.10, 3.11, and 3.12. The v0.3.0 release passed 157 tests, distribution checks, clean-wheel installation, and API/CLI smoke checks.
Non-standard hardware	Not required for installation, API or CLI use, property tests, or reviewer examples. Hardware used for archived analyses is documented in the computational audit.
Examples and reproduction	
Demonstration data	Synthetic regret-locking and tolerance-calibration examples require no restricted data.
Usage instructions	The README documents the Python API, command-line interface, configuration files, checkpoint and resume operation, deterministic locking, and artifact validation.
Expected outputs	Example audit artifacts, schemas, and integrity checks are documented with the reviewer examples.
Reproduction instructions	`REPRODUCIBILITY.md`, manuscript-source documentation, the figure-source manifest, Source Data workbook, and analysis provenance records describe the reproducible boundary. Table S32 has a portable calibration script; Table S33 has an independent verifier covering every protocol-defined bank pair.
Scientific and data boundaries	
Functionality	Main Methods define the objective, candidate rescoring, regret-constrained eligible pool, Jaccard medoid, deterministic tie-breaking, and evaluation boundaries. Supplementary Algorithm 1, propositions, complexity analysis, and the implementation correspondence provide technical detail. The v0.3.0 calibration API is opt-in; the reference locking settings are unchanged.
Restricted data	No participant-level ADNI, AMP-AD, CGGA, provider-organized TCGA workbook, or VGH DICOM-derived records are distributed with the software. Reviewer examples are synthetic.